

MIRAGE FlowPic Classification for Network Traffic Analysis

di Mario Gabriele Carofano

Questo notebook implementa una pipeline completa di **Traffic Classification** con modelli **classici** e **ibridi quantistici** sul dataset MIRAGE, dalla preparazione dei dati fino alla valutazione finale. La pipeline è stata progettata per essere **modulare**, consentendo di scegliere dinamicamente il dataset (grezzo da file .pickle o FlowPic da file .npz), sostituire facilmente i modelli e salvare i risultati in modo organizzato.

Panoramica

Il workflow seguito è:

1. Setup ambiente e riproducibilità

- Import librerie, costanti e funzioni custom.
- Impostazione seed (`random` , `numpy` , `torch`) e scelta automatica del device (`mps` / `cuda` / `cpu`).

2. Caricamento o generazione (opzionale) del dataset

- Il valore della costante `USE_FLOWPIC_DATASET` è una coppia di valori booleani: il primo determina se utilizzare il dataset FlowPic o il dataset grezzo, mentre il secondo determina se caricare dal disco un dataset FlowPic pre-calcolato o generarlo da zero utilizzando le funzioni custom implementate nel modulo `traffic_converter.py` .
- Indipendentemente dalla scelta, il dataset finale può essere identificato dalla variabile `X_raw` (biflussi di traffico) e `y_raw` (etichette).

3. Preparazione dei dati

- **Label Encoding** delle classi di traffico e definizione di `NUM_CLASSES` .
- Split stratificato in **train / validation / test**.
- Riduzione opzionale della dimensione del training set per contenere i tempi di addestramento.
- **Preprocessing (solo per dataset grezzo)**
 - Supporto a più strategie: [`Log1p` , `MinMax` , `MinMax-DirPL` , `Log1p-DirPL`].
 - Aggiornamento dinamico del numero di feature (`N_FEATURES`) in base alla strategia selezionata.
- **Costruzione dataset PyTorch**
 - Dataset custom `MirageDataset` (riordino tensori per CNN1D) o `FlowPicDataset` (per FlowPic).
 - Creazione di `DataLoader` per train, validation e test.

4. Model selection

- Selezione dinamica del modello da `MODEL_REGISTRY` , con supporto a modelli classici e modelli ibridi quantistici (es. `AmplitudeEmbedding` , `AngleEmbedding` , `TrafficCNN` , ecc.).
- Possibilità di recuperare un modello pre-addestrato da checkpoint.

5. Fase di training

- Optimizer: `Adam`
- Loss selezionabile: `CrossEntropy` , `WeightedCrossEntropy` , `Focal`
- Tracking metriche (per epoca).
- Salvataggio best model con **early stopping** opzionale.

6. Fase di valutazione (su test set)

- Calcolo di **accuracy** , **confusion matrix** e **classification report**.

Dataset grezzo (configurabile)

- **Formato**: NumPy arrays serializzati in file pickle
- **Feature per pacchetto**: 4 (DIR, PL, TCPWIN, IAT)
- **Pacchetti massimi per flusso**: 36 (o 100)
- **Pacchetti considerati**: 10 (configurabile)
- **Indicatore di padding**: -1

Dataset di istogrammi FlowPic (configurabile)

- **Formato**: `Tuple[np.ndarray, pd.DataFrame]` , dove:
 - `np.ndarray` : Dataset di istogrammi 2D delle sessioni di traffico, di shape `(N, 1, D, D)` , dove `N` è il numero di finestre temporali valide e `D` è la dimensione dell'istogramma, calcolata in base ai parametri `MTU` e `BIN_SIZE` .
 - `pd.DataFrame` : Metadati dei flussi validi, con le colonne `"FlowID"` , `"DatasetID"` e `"Label"` .
- **Shape dell'input**: `(N, 1, D, D)` — istogrammi 2D FlowPic delle sessioni di traffico.
- **Dimensione dell'istogramma (D)**: Calcolata in base alle costanti `MTU` e `BIN_SIZE` .

Output

Per ogni esperimento, i risultati sono stampati in console e salvati in cartelle create dinamicamente secondo il seguente formato:

`../results/<timestamp>/<model_name>/`. I file prodotti sono:

- `training_history.csv` Storico di training/validation (loss, accuracy, tempi per epoca).
 - `model.pth` Pesi del modello migliore (in base alla validation loss).
 - `train_val_plots.png` Grafici di andamento di loss e accuracy.
 - `confusion_matrix.png` Matrice di confusione normalizzata sul test set.
 - `classification_report.txt` Precision, recall, F1-score e support per classe.
-

Setup ambiente e riproducibilità

```
In [1]: # LIBRARIES
# #####

# Importing constant values
import importlib
import sys

sys.path.insert(1, '../src/')
import constants
importlib.reload(constants)

sys.path.insert(1, '../models/')
import flowpic_models
importlib.reload(flowpic_models)

# Data loading and saving
from traffic_converter import *
from pathlib import Path
import pickle
import os

# Data manipulation and analysis
import numpy as np
import pandas as pd

# Data preprocessing and evaluation
from preprocessing_functions import *
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

# Machine Learning and Quantum ML
from model_selection import *
from training_functions import *
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import pennylane as qml

# Data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Other utilities
import copy
import datetime
import random
import time
```

```
In [2]: # MACROS
# #####

importlib.reload(constants)
from constants import RANDOM_SEED

# #####

# 1. Configurazione del seed per la riproducibilità.
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)
os.environ['PYTHONHASHSEED'] = str(RANDOM_SEED) # Utile per hash di dizionari/set
```

```
# 2. Configurazione di PyTorch (CPU e GPU)
torch.manual_seed(RANDOM_SEED)
torch.cuda.manual_seed(RANDOM_SEED)
torch.cuda.manual_seed_all(RANDOM_SEED)

# 3. Configurazione di PyTorch per la riproducibilità su GPU (CUDNN)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# 4. Configurazione del dispositivo (CPU o GPU)
if torch.backends.mps.is_available():
    DEVICE = torch.device("mps")
elif torch.cuda.is_available():
    DEVICE = torch.device("cuda")
else:
    DEVICE = torch.device("cpu")

print(f"Using device: {DEVICE.type}")
if DEVICE.type == "cuda":
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

Using device: mps

```
In [3]: # CLASSES
# #####

class MirageDataset(Dataset):
    """ Custom PyTorch Dataset per il caricamento e
    la preparazione dei dati di traffico di rete.

    Trasforma i dati da formato (N, Packets, Features)
    a formato (N, Features, Packets) per compatibilità con
    reti neurali convoluzionali 1D (CNN1D).
    """

    def __init__(
        self, X_raw, y_raw,
        device, dtype=None, transform=None,
        lazy_device_transfer=True
    ):
        """Inizializza il dataset Mirage.

        Args:
            X_raw (torch.Tensor): Dati di input, shape (N, Packets, Features).
            y_raw (torch.Tensor): Etichette corrispondenti, shape (N,).

        """

        # MPS non supporta i tensori float64.
        self.device = device
        self.dtype = dtype or (
            torch.float32 if device.type == "mps" else torch.float64
        )

        self.lazy_device_transfer = lazy_device_transfer
        self.transform = transform

        self.X = torch.tensor(X_raw, dtype=self.dtype).permute(0, 2, 1).to(self.device)
        self.y = torch.LongTensor(y_raw).to(self.device)

        # end

    def __len__(self):
        """Ritorna la lunghezza del dataset.

        Returns:
            int: Lunghezza del dataset.

        """

        return len(self.y)

        # end

    def __getitem__(self, idx):
        """Ritorna un elemento del dataset.

        Args:
            idx (int): Indice dell'elemento da recuperare.

        Returns:
            tuple: Coppia (X, y) dell'elemento.

        """

        x = self.X[idx]
        y = self.y[idx]

        if self.transform is not None:
```

```

        x = self.transform(x)

    return x, y

    # end

# end class

class FlowPicDataset(Dataset):
    """ Custom PyTorch Dataset per il caricamento e la preparazione di istogrammi
    2D FlowPic (traffico di rete) e dei relativi metadati.

    A differenza della versione per sequenze di pacchetti (pensata per
    CNN1D), qui i dati sono già nel formato immagine (N, C, H, W)
    adatto a CNN2D senza alcuna trasposizione (o permutazione).

    Caratteristiche principali:
    - Encoding automatico delle etichette testuali (Label) tramite
      sklearn.LabelEncoder, riutilizzabile tra split train/val/test.
    - Trasferimento "lazy" dei tensori sul device per singolo batch,
      per evitare Out-Of-Memory su GPU/MPS con dataset di grandi
      dimensioni (es. 93300 x 1 x 150 x 150 ~ 8-17 GB in RAM).
    - Accesso ai metadati originali (FlowID, DatasetID, Label) per
      ogni campione, utile per debug/analisi degli errori.
    - Supporto opzionale per trasformazioni (normalizzazione, augmentation).
    """

    def __init__(
        self, X_raw, y_raw,
        device=None, dtype=None, transform=None,
        lazy_device_transfer=True
    ):
        """Inizializza il dataset FlowPic.

        Args:
            X_raw (np.ndarray): Istogrammi 2D, shape (N, 1, H, W).
            y_raw (np.ndarray): Etichette già codificate numericamente, shape (N,).
            device (torch.device): Device usato per il trasferimento lazy dei batch.
            dtype (torch.dtype, optional): Tipo dei tensori immagine.
                Se None, float32 su MPS (che non supporta float64),
                float64 altrove. Defaults to None.
            transform (Callable, optional): Funzione applicata a ciascun tensore
                immagine in __getitem__ (es. normalizzazione). Defaults to None.
            lazy_device_transfer (bool, optional): Se True, i dati restano in CPU
                (pinnati se CUDA) e vengono spostati sul device solo per il batch
                richiesto, evitando OOM su GPU/MPS. Defaults to True.
        """

        super().__init__()

        if len(X_raw) != len(y_raw):
            raise ValueError(
                f"X_raw ({len(X_raw)}) e y_raw ({len(y_raw)}) devono avere "
                f"lo stesso numero di campioni."
            )

        # MPS non supporta i tensori float64.
        self.device = device
        self.dtype = dtype or (
            torch.float32 if device.type == "mps" else torch.float64
        )

        self.lazy_device_transfer = lazy_device_transfer
        self.transform = transform

        X_t = torch.from_numpy(np.asarray(X_raw)).to(self.dtype).contiguous()
        y_t = torch.from_numpy(np.asarray(y_raw)).long()

        if self.lazy_device_transfer:
            # Tiene i dati in CPU; il trasferimento avviene per batch in __getitem__.
            self.X = X_t.pin_memory() if self.device.type == "cuda" else X_t
            self.y = y_t
        else:
            # Trasferisce tutto sul device (attenzione a OOM su GPU/MPS).
            self.X = X_t.to(self.device, non_blocking=True)
            self.y = y_t.to(self.device, non_blocking=True)

    # end

    def __len__(self):
        """Ritorna la lunghezza del dataset.

        Returns:
            int: Numero di campioni nel dataset.
        """

```

```

        return self.y.shape[0]

    # end

def __getitem__(self, idx):
    """Ritorna un elemento del dataset, spostato sul device se
    richiesto dal trasferimento lazy.

    Args:
        idx (int): Indice dell'elemento da recuperare.

    Returns:
        tuple: Coppia (X, y) del campione richiesto.
    """
    x = self.X[idx]
    y = self.y[idx]

    if self.transform is not None:
        x = self.transform(x)

    if self.lazy_device_transfer:
        x = x.to(self.device, non_blocking=True)
        y = y.to(self.device, non_blocking=True)

    return x, y

    # end

# end class

```

```

In [54]: # FUNCTIONS
# #####

def flow_debug(
    debug: bool = False,
    flows_to_inspect: list = None,
    metadata: pd.DataFrame = None,
    histograms: np.ndarray = None
) -> None:

    """Stampa informazioni di debug sui flussi specificati, se il flag di debug è attivo.
    Mostra le coordinate degli elementi non nulli, i valori e le statistiche di base
    come min, max, media, deviazione standard e somma.

    Args:
        debug (bool): Flag per abilitare la stampa delle informazioni di debug.
        flows_to_inspect (list): Lista degli ID dei flussi da ispezionare.
        metadata (pd.DataFrame): DataFrame contenente i metadati dei flussi.
        histograms (np.ndarray): Array NumPy contenente gli istogrammi 2D (FlowPic).
    """

    if debug and flows_to_inspect is not None:

        for fid in flows_to_inspect:

            flow_metadata = metadata.loc[metadata['FlowID'] == fid]

            if flow_metadata.empty:
                print(f"[DEBUG] Flusso n.{fid} non trovato.")
                continue

            did = flow_metadata['DatasetID'].values[0]
            flow = histograms[did][0]

            # #####
            #     Stampa di shape e metadati del flusso

            print(f"[DEBUG] Elaborazione del flusso n.{fid}")
            print(f"Shape: {histograms[did].shape}")
            print(f"Metadata:\n{flow_metadata.to_string(index=False)}")

            # #####
            #     Stampa degli elementi non nulli
            #     Mostra le coordinate (riga, colonna) e il valore corrispondente.

            rows, cols = np.nonzero(flow)
            values = flow[rows, cols]

            print("Coordinate degli elementi non nulli:")
            for r, c, v in zip(rows, cols, values):
                print(f"{{r}},{{c}} = {{v}}", end=" | ")

            print()

            # #####

```

```

#         Stampa delle statistiche di base

print(f"Min: {np.min(histograms[did])}")
print(f"Max: {np.max(histograms[did])}")
print(f"Mean: {np.mean(histograms[did])}")
print(f"Std: {np.std(histograms[did])}")
print(f"Sum: {np.sum(histograms[did])}\n")

# end for fid
# end if

# end

def get_flowpic_dir(data_path: str) -> str:
    """ Sostituisce il nome 'flowpic' al posto di 'mirage' nel percorso del dataset. """

    p = Path(data_path)
    parts = p.parts

    if "mirage" not in parts:
        raise ValueError(
            f"Il percorso '{data_path}' non contiene la cartella 'mirage'.\n"
            "Assicurati di fornire un percorso valido che contenga 'mirage'."
        )

    idx = parts.index("mirage")
    new_parts = list(parts)
    new_parts[idx] = "flowpic"

    return str(Path(*new_parts))

# end

def get_plot_epoch_ticks(
    num_epochs : int,
    best_epoch_num: int
) -> tuple[np.ndarray, list[str]]:
    """Genera i tick per per l'asse X di un grafico
    che mostra l'andamento di loss/accuracy in funzione delle epoche.

    I tick includono sempre:
    - la prima epoca;
    - la miglior epoca;
    - L'ultima epoca;
    - ulteriori valori intermedi uniformemente distribuiti.

    Args:
        num_epochs (int): Numero totale di epoche.
        best_epoch_num (int): Numero dell'epoca con la migliore performance.

    Returns:
        tuple: Una tupla contenente gli indici dei tick e le etichette corrispondenti.
    """

    # Il totale numero di tick da visualizzare è limitato a 10.
    num_ticks = min(10, num_epochs)

    # Si distribuiscono uniformemente i tick,
    # includendo sempre la prima e l'ultima epoca.
    tick_indices = np.linspace(
        start=1, stop=num_epochs,
        num=num_ticks,
        dtype=int
    )

    # Si inserisce manualmente l'epoca con la migliore performance.
    tick_indices = np.unique(
        np.append(tick_indices, best_epoch_num)
    )

    # Se l'inserimento della best epoch porta a più di 10 tick,
    # si rimuove il tick intermedio più vicino alla best epoch,
    # mantenendo sempre la prima, la best e l'ultima epoca.
    while len(tick_indices) > num_ticks:
        removable = [
            tick for tick in tick_indices
            if tick not in {1, best_epoch_num, num_epochs}
        ]

        if not removable:
            break

        tick_to_remove = min(
            removable,
            key=lambda tick: abs(tick - best_epoch_num)
        )

```

```

        tick_indices = tick_indices[tick_indices != tick_to_remove]

        # end while

    tick_indices = np.sort(tick_indices)
    tick_labels = [str(epoch) for epoch in tick_indices]

    return tick_indices, tick_labels

# end

def get_plot_metric_ticks(
    min_value : float,
    max_value : float,
    best_value : float,
) -> tuple[np.ndarray, list[str]]:
    """Genera i tick per per l'asse Y di un grafico
    che mostra l'andamento di loss/accuracy in funzione delle epoche.

    I tick includono sempre:
    - il valore minimo della metrica;
    - il valore della metrica nella best epoch;
    - il valore massimo della metrica;
    - ulteriori valori intermedi uniformemente distribuiti.

    Args:
        metric_history (list[float]): Valori storici della metrica, uno per epoca.
        best_epoch_num (int): Numero dell'epoca migliore, in formato 1-based.

    Returns:
        tuple[np.ndarray, list[str]]: Array dei valori dei tick e relative
        etichette formattate.
    """

    # Caso limite: tutti i valori della metrica sono vicinissimi.
    if np.isclose(min_value, max_value):
        tick_values = np.array([min_value])
        tick_labels = [f"{min_value:.4f}"]

        return tick_values, tick_labels

    # Il totale numero di tick da visualizzare è limitato a 10,
    # uniformemente distribuiti tra minimo e massimo.
    max_num_ticks = 10
    tick_values = np.linspace(
        start=min_value,
        stop=max_value,
        num=max_num_ticks
    )

    # Si inserisce il valore della metrica relativo alla best epoch.
    tick_values = np.unique(
        np.append(tick_values, best_value)
    )

    # Se sono presenti più di 10 tick, rimuove quello intermedio
    # più vicino al valore della best epoch, preservando minimo,
    # valore best epoch e massimo.
    while len(tick_values) > max_num_ticks:
        removable = [
            tick for tick in tick_values
            if not np.isclose(tick, min_value)
            and not np.isclose(tick, best_value)
            and not np.isclose(tick, max_value)
        ]

        if not removable:
            break

        tick_to_remove = min(
            removable,
            key=lambda tick: abs(tick - best_value)
        )

        tick_values = tick_values[~np.isclose(tick_values, tick_to_remove)]

    # end while

    # Si ordinano i tick in ordine crescente.
    tick_values = np.sort(tick_values)

    # Per loss e accuracy, 4 decimali costituiscono un buon default.
    tick_labels = [f"{tick:.4f}" for tick in tick_values]

    return tick_values, tick_labels

```

```

# end

def draw_train_val_plot(
    p : plt.Axes,
    metric_name : str,
    train_values: list[float],
    val_values: list[float],
    best_epoch: int
) -> None:

    num_epochs = len(train_values)
    epochs_range = range(1, num_epochs + 1)
    best_value = float(val_values[best_epoch - 1])

    p.plot(epochs_range, train_values, label=f'Train {metric_name}')
    p.plot(epochs_range, val_values, label=f'Val {metric_name}')
    p.set_title(f'{metric_name} over Epochs')
    p.set_xlabel('Epoch')
    p.set_ylabel(metric_name)
    p.legend()

    p.axvline(
        best_epoch,
        color='red', linestyle='--', dashes=(10, 15), linewidth=0.7,
        # label=f'Best epoch ({best_epoch})'
    )

    p.axhline(
        best_value,
        color='red', linestyle='--', dashes=(10, 15), linewidth=0.7,
        # label=f'Best epoch ({best_epoch})'
    )

    epoch_indices, epoch_labels = get_plot_epoch_ticks(num_epochs, best_epoch)

    p.set_xticks(epoch_indices)
    p.set_xticklabels(epoch_labels)
    p.set_xlim(0.5, num_epochs + 0.5)

    metric_indices, metric_labels = get_plot_metric_ticks(
        float(np.min(train_values + val_values)),
        float(np.max(train_values + val_values)),
        best_value
    )

    margin = (metric_indices[-1] - metric_indices[0]) * 0.05
    p.set_yticks(metric_indices)
    p.set_yticklabels(metric_labels)
    p.set_ylim(metric_indices[0] - margin, metric_indices[-1] + margin)

# end

```

Caricamento o generazione (opzionale) del dataset

```

In [5]: importlib.reload(constants)
from constants import (
    USE_FLOWPIC_DATASET,
    DATA_PATH, DATASET_NAME,
    MIN_TPS, MIN_PACKETS, MIN_DIM
)

# #####

# Si impostano le variabili per la scelta del dataset da utilizzare.
use_flowpic, use_precomputed = USE_FLOWPIC_DATASET

# Si impostano i filtri per la selezione dei flussi da elaborare.
filters = {
    'min_tps': MIN_TPS,
    # 'min_dim': MIN_DIM,
    # 'min_packets': MIN_PACKETS,
}

# Si impostano le variabili di debug per il caricamento del dataset.
debug = False
debug_cycle = False
flows_to_inspect = None

# #####

print("Loading dataset...")

```



```

print(f"DEBUG impostato su {debug}.")
print(f"USE_FLOWPIC_DATASET impostato su {use_flowpic}.")
print(f"USE_PRECOMPUTED_DATASET impostato su {use_precomputed}.\n")

if not use_flowpic:

    # Il file pickle contiene due oggetti salvati in sequenza:
    # 1. X_raw : i dati numerici dei flussi di traffico, in formato numpy array.
    # 2. y_raw : le etichette corrispondenti ai flussi.
    # Ogni chiamata a pickle.load() legge il successivo oggetto nel file.
    with open(f"{DATA_PATH}/{DATASET_NAME}", "rb") as f:
        X_raw = np.array(pickle.load(f), dtype=np.float32)
        y_raw = np.array(pickle.load(f))

    if debug:
        example_sample = 7000
        print("\nExample Sample (First 5 packets):\n", X_raw[example_sample][:5])
        print("Example Label:", y_raw[example_sample])

elif use_flowpic:

    flowpics_name = get_dataset_name(DATA_PATH, DATASET_NAME, debug)
    flowpics_dir = get_flowpic_dir(DATA_PATH)
    os.makedirs(flowpics_dir, exist_ok=True)

    npz_path = os.path.join(flowpics_dir, f"{flowpics_name}.npz")
    meta_path = os.path.join(flowpics_dir, f"{flowpics_name}_metadata.csv")

    if not use_precomputed:

        histograms, metadata = mirage_pickle_converter(
            f"{DATA_PATH}/{DATASET_NAME}",
            filters, debug, debug_cycle, flows_to_inspect
        )

        # Se il flag di debug è attivo,
        # si stampano informazioni dettagliate sui flussi specificati.
        flow_debug(debug, flows_to_inspect, metadata, histograms)

        # Salvataggio degli istogrammi 2D FlowPic in formato NumPy.
        np.savez_compressed(npz_path, data=histograms)
        print(f"Salvato dataset (shape={histograms.shape}) in: {npz_path}")

        # Salvataggio dei metadati in formato CSV.
        metadata.to_csv(meta_path, index=False)
        print(f"Salvati metadati (di {len(metadata)} righe) in: {meta_path} ")

    elif use_precomputed:
        if not os.path.exists(npz_path):
            raise FileNotFoundError(f"Array di istogrammi precomputati non trovato: {npz_path}")
        if not os.path.exists(meta_path):
            raise FileNotFoundError(f"DataFrame di metadati precomputati non trovato: {meta_path}")

        # Caricamento degli istogrammi 2D FlowPic precomputati.
        histograms = np.load(npz_path)['data']
        print(f"Caricato dataset (shape={histograms.shape}) da: {npz_path}")

        # Caricamento dei metadati in formato CSV.
        metadata = pd.read_csv(meta_path)
        print(f"Caricati metadati (di {len(metadata)} righe) da: {meta_path} ")

        # end if "use_precomputed"

    # Prima di procedere con split/training, si verifica che gli istogrammi e i metadati siano allineati.
    assert histograms.shape[0] == len(metadata), "Disallineamento tra istogrammi e metadati!"
    assert list(metadata["DatasetID"]) == list(range(len(metadata))), "DatasetID non contiguo!"

    # Per coerenza con l'altro notebook, si rinominano le variabili per il dataset e le etichette.
    X_raw = histograms
    y_raw = metadata["Label"].values

    # end if "use_flowpic"

```

Loading dataset...

DEBUG impostato su False.

USE_FLOWPIC_DATASET impostato su True.

USE_PRECOMPUTED_DATASET impostato su True.

Caricato dataset (shape=(93300, 1, 150, 150)) da: ../dataset/flowpic/2019/mirage2019_100P_4F_APP_PAD.npz

Caricati metadati (di 93300 righe) da: ../dataset/flowpic/2019/mirage2019_100P_4F_APP_PAD_metadata.csv

Preparazione dei dati

Label Encoding

In questo blocco di codice, le etichette categoriche vengono trasformate in valori numerici utilizzando `LabelEncoder`. Ogni classe di traffico viene associata a un identificativo numerico, facilitando l'utilizzo nei modelli ML.

Le classi originali vengono memorizzate in `DATASET_CLASSES` per riferimenti futuri, mentre il numero totale di classi viene salvato in `NUM_CLASSES`.

```
In [6]: le = LabelEncoder()

y_encoded = le.fit_transform(y_raw)

DATASET_CLASSES = le.classes_
""" List of unique classes in the dataset, determined by the unique labels in y_raw after encoding. """

NUM_CLASSES = len(le.classes_)
""" Number of unique classes in the dataset. """

print(
    f"Number of classes: {NUM_CLASSES} \n\n" +
    f"Classes: {DATASET_CLASSES}"
)
```

Number of classes: 40

Classes: ['8 Ball Pool' 'AccuWeather' 'Amazon' 'Booking' 'Comics' 'Diretta' 'Dropbox' 'Duolingo' 'Facebook' 'Flipboard' 'FourSquare' 'Google Drive' 'Groupon' 'Hangouts' 'Instagram' 'Linkedin' 'Messenger' 'Musixmatch' 'OneDrive' 'OneFootball' 'Pinterest' 'Playstore' 'Reddit' 'Skype' 'Slither.io' 'SoundCloud' 'Spotify' 'Subito' 'Telegram' 'Trello' 'TripAdvisor' 'Tumblr' 'Twitter' 'Uber' 'Viber' 'Waze' 'Wish' 'Youtube' 'eBay' 'ilMeteo']

Train-Validation-Test split

Il dataset viene suddiviso in tre insiemi distinti mediante doppio split stratificato, le cui dimensioni sono specificate dalle costanti `TRAIN_SIZE`, `VAL_SIZE` e `TEST_SIZE`, preservando la distribuzione originale delle classi.

Se il flag `USE_NEW_SIZE` è attivato e il training set supera `NEW_TRAIN_SIZE` campioni, il training set viene ridotto tramite campionamento casuale per contenere i tempi di addestramento.

```
In [7]: importlib.reload(constants)
from constants import (
    TRAIN_SIZE, VAL_SIZE, TEST_SIZE,
    USE_NEW_SIZE, NEW_DATASET_SIZE,
    RANDOM_SEED
)

# #####

# Verifica che le proporzioni di Train, Val e Test sommino a "1".
assert TRAIN_SIZE + VAL_SIZE + TEST_SIZE == 1.0, \
    "Le percentuali di Train, Val e Test devono sommare a 1."

# Se il flag USE_NEW_SIZE è attivo, si riduce il dataset a NEW_DATASET_SIZE campioni.
if USE_NEW_SIZE and len(X_raw) > NEW_DATASET_SIZE:
    X_raw, _, y_encoded, _ = train_test_split(
        X_raw, y_encoded,
        train_size=NEW_DATASET_SIZE,
        stratify=y_encoded,
        random_state=RANDOM_SEED
    )

    # end if USE_NEW_SIZE

n_total = len(X_raw)

# Il primo split divide il dataset in due parti: Train+Val e Test.
X_temp, X_test, y_temp, y_test = train_test_split(
    X_raw, y_encoded,
    test_size=TEST_SIZE,
    stratify=y_encoded,
    random_state=RANDOM_SEED
)

# Il secondo split divide il Train+Val rimanente in due insiemi: Train e Val.
val_fraction_of_temp = VAL_SIZE / (TRAIN_SIZE + VAL_SIZE)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp,
    test_size=val_fraction_of_temp,
    stratify=y_temp,
    random_state=RANDOM_SEED
)
```

```

)

assert len(X_train) + len(X_val) + len(X_test) == n_total, \
    "La somma dei campioni Train, Val e Test non corrisponde al totale!"

print(
    f"Total samples: {n_total} \n" +
    f"Train shape: {X_train.shape} ({len(X_train)/n_total:.1%}) \n" +
    f"Val shape: {X_val.shape} ({len(X_val)/n_total:.1%}) \n" +
    f"Test shape: {X_test.shape} ({len(X_test)/n_total:.1%}) \n"
)

```

```

Total samples: 30000
Train shape: (21299, 1, 150, 150) (71.0%)
Val shape: (4201, 1, 150, 150) (14.0%)
Test shape: (4500, 1, 150, 150) (15.0%)

```

Preprocessing (solo per dataset grezzo)

In questa sezione vengono raccolte le principali strategie di preprocessing applicate ai biflussi del dataset prima della fase di training. L'obiettivo è trasformare i dati grezzi in una rappresentazione più adatta ai modelli classici e ibridi quantistici. Queste strategie consentono di confrontare diverse modalità di preparazione dei dati, da approcci più semplici e generici a soluzioni più mirate al dominio del traffico di rete.

Le feature originali considerate sono:

- **DIR** : direzione del pacchetto
- **PL** : packet length
- **TCPWIN** : finestra TCP
- **IAT** : inter-arrival time

Le strategie di preprocessing implementate sono:

1. Masking del padding + Log1p normalization: (Log1p)

Approccio guidato dal dominio applicativo. I pacchetti di padding vengono identificati e azzerati per evitare che influenzino il modello. Successivamente viene applicata una trasformazione **Log1p** alle feature numeriche (**PL** , **TCPWIN** , **IAT**) per comprimere il range dinamico e ridurre l'effetto degli outlier.

2. Min-Max Scaling standard: (MinMax)

Approccio generico che applica una normalizzazione lineare nell'intervallo **[0, 1]** a tutte le feature. Non gestendo esplicitamente il padding, questa strategia può trattare i valori fittizi come dati reali e risultare sensibile agli outlier.

3. Fusione DIR/PL + Min-Max Scaling: (MinMax-DirPL)

La direzione (**DIR**) e la lunghezza (**PL**) vengono combinate in una singola feature con segno, così da rappresentare il traffico in modo più compatto. Dopo questa trasformazione, il numero di feature passa da 4 a 3 e viene applicato un Min-Max Scaling standard.

4. Masking del padding + Log1p + fusione DIR/PL: (Log1p-DirPL)

Strategia ibrida che unisce i vantaggi della Strategy 1 e della Strategy 3. Prima gestisce correttamente il padding e applica **Log1p**, poi combina **DIR** e **PL** in una feature con segno. In questo modo si ottiene una rappresentazione più compatta, semanticamente coerente e generalmente più robusta.

```

In [8]: PREPROCESSING_REGISTRY = {
        "Log1p": log1pPreprocessing,
        "MinMax": minMaxPreprocessing,
        "MinMax-DirPL": lambda X: minMaxPreprocessing(X, combine_dir_pl_flag=True),
        "Log1p-DirPL": lambda X: log1pPreprocessing(X, combine_dir_pl_flag=True),
    }

    """ Specifies the preprocessing strategy to apply to the dataset. """

# #####

# Si definisce la strategia di preprocessing da applicare ai dati.
PREPROCESSING_STRATEGY = "MinMax"

if PREPROCESSING_STRATEGY not in PREPROCESSING_REGISTRY:
    raise ValueError(
        f"PREPROCESSING_STRATEGY deve essere uno tra: " +
        f"{list(PREPROCESSING_REGISTRY.keys())}"
    )

preprocess_fn = PREPROCESSING_REGISTRY[PREPROCESSING_STRATEGY]

# #####

```

```

print(f"USE_FLOWPIC_DATASET impostato su {use_flowpic}.\n")
if use_flowpic:
    print("La fase di preprocessing è disponibile solo per il dataset grezzo (Mirage).")
    pass

elif not use_flowpic:

    print(f"Applying preprocessing strategy {PREPROCESSING_STRATEGY}...")

    X_train_proc = preprocess_fn(X_train)
    X_val_proc = preprocess_fn(X_val)
    X_test_proc = preprocess_fn(X_test)

    # Aggiorna il numero di feature in base alla strategia scelta.
    N_FEATURES = X_train_proc.shape[2]

    print("\nPreprocessing complete.")

    print(f"\nTrain shape: {X_train_proc.shape}")
    print(f"Val shape: {X_val_proc.shape}")
    print(f"Test shape: {X_test_proc.shape}")

    print(f"\nAdjusted N_FEATURES: {N_FEATURES}")

```

USE_FLOWPIC_DATASET impostato su True.

La fase di preprocessing è disponibile solo per il dataset grezzo (Mirage).

Costruzione dataset PyTorch

Un `DataLoader` è un oggetto che preleva campioni dal dataset e genera batch in modo efficiente.

Gli iperparametri principali del `DataLoader` sono:

- **Batch size** (cioè il numero di campioni in un mini-batch).

Utilizzando la GPU, un batch size più grande rende l'addestramento più efficiente. Tuttavia, un batch size più piccolo può portare a risultati migliori in termini di accuratezza finale. **La selezione del batch size appropriato e di altri iperparametri è fondamentale per l'ottimizzazione del modello e dipende dalle caratteristiche specifiche del dataset e dell'architettura del modello.**

- **Number of workers**

Questo iperparametro determina il numero di processi paralleli utilizzati per caricare i dati. Un numero maggiore di worker può accelerare il caricamento dei dati, ma può anche aumentare l'uso della memoria e la complessità del sistema. **È buona norma impostarli in base al numero di core della CPU.**

- **Shuffle**

Se impostato su `True`, i dati vengono mescolati ad ogni epoca, garantendo che il modello non impari sequenze specifiche dei dati. Si può impostare questo iperparametro per evitare overfitting e migliorare la generalizzazione del modello (solo per la fase di training).

- **Drop last**

Se impostato su `True`, l'ultimo batch viene scartato se non contiene il numero completo di campioni. Questo può essere utile per garantire che tutti i batch abbiano la stessa dimensione, semplificando l'addestramento del modello. **Si consiglia di impostarlo su `True` per il training e su `False` per la validazione e il test, perché in questi ultimi casi è importante valutare il modello su tutti i campioni disponibili.**

```

In [9]: importlib.reload(constants)
from constants import BATCH_SIZE

# #####

print(f"USE_FLOWPIC_DATASET impostato su {use_flowpic}.\n")
if use_flowpic:
    # Si utilizza la classe dedicata FlowPicDataset per creare i dataset di PyTorch.
    train_dataset = FlowPicDataset(X_train, y_train, device=DEVICE)
    val_dataset = FlowPicDataset(X_val, y_val, device=DEVICE)
    test_dataset = FlowPicDataset(X_test, y_test, device=DEVICE)

elif not use_flowpic:
    # Si utilizza la classe dedicata MirageDataset per creare i dataset di PyTorch,
    # in modo che siano compatibili con le reti neurali convoluzionali 1D (CNN1D).
    train_dataset = MirageDataset(X_train_proc, y_train, device=DEVICE)
    val_dataset = MirageDataset(X_val_proc, y_val, device=DEVICE)
    test_dataset = MirageDataset(X_test_proc, y_test, device=DEVICE)

```

```
# ##### #

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    num_workers=0,
    shuffle=True, drop_last=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    num_workers=0,
    shuffle=False, drop_last=False
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    num_workers=0,
    shuffle=False, drop_last=False
)
```

USE_FLOWPIC_DATASET impostato su True.

Model Selection

In base alla modalità di esecuzione selezionata (`EXEC_MODE_TRAIN`), il modello viene istanziato con i parametri appropriati.

- **Fase di training** (`EXEC_MODE_TRAIN` impostato su `True`):

Il modello viene addestrato da zero.

- **Fase di test** (`EXEC_MODE_TRAIN` impostato su `False`):

Il modello viene caricato da un file di pesi precedentemente addestrato e salvato. Inoltre, viene anche caricato il suo storico di addestramento. Il modello viene istanziato con i parametri appropriati, e i pesi salvati vengono caricati nel modello.

I modelli disponibili sono definiti nei seguenti file, e comprendono sia modelli classici che modelli ibridi quantistici:

da `models/nn_models.py`

- **Amplitude Embedding Model** (`AmplitudeEmbedding`)

Una rete neurale ibrida che sfrutta l'Amplitude Embedding per codificare i dati classici nelle ampiezze dello stato quantistico. Questo approccio massimizza la densità dei dati, consentendo la codifica di 2^N features in N qubit, preceduta da uno strato classico attivato da una funzione sigmoide.

- **Angle Embedding Model** (`AngleEmbedding`)

Un modello ibrido semplificato che utilizza l'Angle Embedding, dove le feature di input vengono mappate direttamente sugli angoli di rotazione dei qubit in un rapporto 1:1. Presenta uno strato di pre-elaborazione classico seguito da un circuito quantistico con strati fortemente entangled, offrendo una strategia di embedding shallow e resistente al rumore.

- **Ring Model** (`RingEmbedding`)

Un'architettura ibrida che impiega una strategia di Ring Embedding personalizzata, suddividendo l'input in due set di feature codificate tramite rotazioni e schemi di entanglement circolari CNOT. Questo design raddoppia la capacità dei dati rispetto al semplice angle embedding e introduce correlazioni tra i qubit già dalle prime fasi del circuito.

- **Waterfall Model** (`WaterfallEmbedding`)

Un modello ibrido complesso che presenta uno schema di Waterfall Embedding, suddividendo gli input in blocchi di rotazione Y e Z. Integra una connettività densa e all-to-all di porte CNOT nella prima fase, creando uno stato altamente entangled prima degli strati del variational ansatz.

- **Classical 1D CNN Model** (`TrafficCNN`)

Un modello classico basato su una rete neurale convoluzionale 1D, progettata per estrarre caratteristiche locali dai dati sequenziali. Questo approccio sfrutta strati convoluzionali e di pooling per catturare pattern temporali o spaziali nei dati di input.

da `models/complex_hybrid_models.py`

- **AmpCnn Model** (`AmpCnn`)

Un modello ibrido che combina l'Amplitude Embedding con una rete neurale convoluzionale 1D, sfruttando le capacità di codifica quantistica per migliorare l'estrazione delle caratteristiche locali dai dati sequenziali.

- **Classical Twin Model** (`ClassicalTwin`)

Un modello classico basato su una rete neurale a due rami, progettata per elaborare due flussi di dati paralleli e combinare le informazioni estratte per migliorare le prestazioni predittive.

- **Classical Light Model** (`ClassicalLight`)

Un modello classico leggero, progettato per essere efficiente in termini di risorse computazionali e memoria, pur mantenendo buone prestazioni predittive.

- **CnnAmpCnn Model** (`CnnAmpCnn`)

Un modello ibrido **CNN–Quantum–CNN**: una prima CNN 1D estrae feature locali dai pacchetti, che vengono proiettate (con layer denso + sigmoide) nello spazio richiesto dall'**Amplitude Embedding**. L'output del circuito quantistico (con strati fortemente entangled) viene poi raffinato da una seconda CNN 1D e da layer fully connected per la classificazione finale multiclasse.

- **Dense Model** (`Dense`)

Un modello classico basato su una rete neurale fully connected, progettato per elaborare input di dimensioni fisse e catturare relazioni complesse tra le feature, grazie a strati densi e funzioni di attivazione non lineari.

da `models/flowpic_models.py`

- **FlowPicCNN Model** (`FlowPicCNN`)

Un modello classico basato su una rete neurale convoluzionale 2D, progettata per elaborare immagini di istogrammi FlowPic. Questo approccio sfrutta strati convoluzionali e di pooling per catturare pattern spaziali nei dati di input, ottimizzando la classificazione del traffico di rete.

- ...

```
In [10]: importlib.reload(constants)
from constants import (
    EXEC_MODE_TRAIN,
    OUTPUT_DIR, MODEL_TIMESTAMP_ID, MODEL_NAME,
    N_QUBITS, N_LAYERS,
    N_FEATURES, N_PACKETS
)

importlib.reload(flowpic_models)
from flowpic_models import ResidualBlock

# #####

SELECTED_MODEL = "ResNet" if use_flowpic else "TrafficCNN"

HybridModel = get_model_class(SELECTED_MODEL)

# model = HybridModel(
#     n_qubits=N_QUBITS,
#     n_layers=N_LAYERS,
#     n_features=N_FEATURES,
#     n_packets=N_PACKETS,
#     num_classes=NUM_CLASSES
# )

model = HybridModel(
    block=ResidualBlock,
    layers=[1, 2, 3, 1],
    num_classes=NUM_CLASSES
)

# #####

# Si sposta il modello sul dispositivo corretto (CPU, GPU o MPS)
# e si imposta il tipo di dato appropriato.
model = model.to(DEVICE).float() if DEVICE.type == 'mps' else model.to(DEVICE).double()

# Inizializzazione dei pesi del modello.
if debug: help(weight_init)
model.apply(weight_init)

print(f"EXEC_MODE_TRAIN impostato su {EXEC_MODE_TRAIN}.\n")
if EXEC_MODE_TRAIN:
    print(f"Modello selezionato: {SELECTED_MODEL} -> {HybridModel.__name__}")
```

```

print(model.get_model_name())
print(model)

total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Totale parametri addestrabili: {total_params:,}")

elif not EXEC_MODE_TRAIN:
    print(f"Modello selezionato: {MODEL_TIMESTAMP_ID} • {MODEL_NAME}")
    print("Il modello non verrà addestrato, ma solo valutato sul test set.")

    # Si caricano i pesi salvati del modello dalla directory di output specificata.
    model_dir = f"{OUTPUT_DIR}/{MODEL_TIMESTAMP_ID}/{MODEL_NAME}/model.pth"
    weights = torch.load(model_dir, map_location=DEVICE)

    # Verifica la compatibilità tra il modello corrente e i pesi salvati.
    check_model_compatibility(model, weights)

    # Carica i pesi nel modello corrente e stampa l'esito del caricamento.
    load_result = model.load_state_dict(weights)
    print(f"\nEsito caricamento: {load_result}")
    print("Model loaded correctly.")

    # Si imposta il modello in modalità di valutazione
    # per disabilitare il dropout e la batch normalization.
    model.eval()

    # Si recupera lo storico delle metriche di addestramento e validazione dal file CSV salvato.
    df_history = pd.read_csv(f"{OUTPUT_DIR}/{MODEL_TIMESTAMP_ID}/{MODEL_NAME}/training_history.csv")
    history = df_history.to_dict(orient='list')

# end if

```

EXEC_MODE_TRAIN impostato su True.

Modello selezionato: ResNet -> ResNet

ResNet_C40

```
ResNet(  
  (conv1): Sequential(  
    (0): Conv2d(1, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3))  
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)  
  )  
  (layer0): Sequential(  
    (0): ResidualBlock(  
      (conv1): Sequential(  
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
      )  
      (conv2): Sequential(  
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (relu): ReLU(inplace=True)  
    )  
  )  
  (layer1): Sequential(  
    (0): ResidualBlock(  
      (conv1): Sequential(  
        (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
      )  
      (conv2): Sequential(  
        (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (skip_module): Sequential(  
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2))  
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (relu): ReLU(inplace=True)  
    )  
    (1): ResidualBlock(  
      (conv1): Sequential(  
        (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
      )  
      (conv2): Sequential(  
        (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (relu): ReLU(inplace=True)  
    )  
  )  
  (layer2): Sequential(  
    (0): ResidualBlock(  
      (conv1): Sequential(  
        (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
      )  
      (conv2): Sequential(  
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (skip_module): Sequential(  
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2))  
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (relu): ReLU(inplace=True)  
    )  
    (1): ResidualBlock(  
      (conv1): Sequential(  
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
      )  
      (conv2): Sequential(  
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (relu): ReLU(inplace=True)  
    )  
    (2): ResidualBlock(  
      (conv1): Sequential(  

```



```

        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
    )
    (conv2): Sequential(
      (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (relu): ReLU(inplace=True)
  )
)
(layer3): Sequential(
  (0): ResidualBlock(
    (conv1): Sequential(
      (0): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU(inplace=True)
    )
    (conv2): Sequential(
      (0): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (skip_module): Sequential(
      (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2))
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (relu): ReLU(inplace=True)
  )
)
(clf): Sequential(
  (0): AdaptiveAvgPool2d(output_size=1)
  (1): Flatten(start_dim=1, end_dim=-1)
  (2): Linear(in_features=512, out_features=40, bias=True)
)
)

```

Totale parametri addestrabili: 7,580,968

Fase di training

...

- **Optimizer:** è l'algoritmo che aggiorna i pesi del modello dopo ogni batch, usando i gradienti calcolati con la backpropagation (e.g. in pratica decide come e quanto modificare i parametri per ridurre l'errore). Pertanto, la scelta dell'optimizer (e del **learning rate**) influenza stabilità, velocità di convergenza e qualità finale delle predizioni.

Le scelte più comuni sono:

- **SGD**

Ottimizzatore "classico": aggiorna i pesi con passi uniformi (spesso con **momentum**).

È adatto quando si vuole un comportamento semplice e controllabile, e funziona bene su modelli/dataset stabili (con tuning accurato del learning rate).

- **Adam**

In generale, **Adam** è preferito come scelta di default perché usa learning rate adattivi e converge più rapidamente su problemi complessi o con poco tuning.

- **RMSprop**, **AdamW**

RMSprop è spesso utile con gradienti rumorosi/non stazionari (es. dati sequenziali), mentre **AdamW** è indicato quando si vuole una regolarizzazione migliore grazie al weight decay disaccoppiato.

- **Scheduler:** è un meccanismo che modifica dinamicamente il learning rate durante l'addestramento, in base a una strategia predefinita. L'obiettivo è migliorare la convergenza e la stabilità del training, evitando di rimanere bloccati in minimi locali o oscillazioni eccessive.

Le strategie disponibili includono:

- **OneCycleSched**

Implementa la strategia **One Cycle Policy**, che aumenta il learning rate fino a un massimo e poi lo riduce, con un ciclo di warm-up e cool-down. Questo approccio può accelerare la convergenza e migliorare le prestazioni finali del modello.

- **LinearSched**

Implementa una riduzione lineare del learning rate durante l'addestramento, partendo da un valore iniziale e scendendo fino a un valore finale. È utile quando si desidera un controllo più semplice e prevedibile sul learning rate.

- **StepSched**

Implementa una riduzione a gradini del learning rate, diminuendolo di un fattore `gamma` specificato ogni `step_size` epoche. Questo approccio può essere utile quando si desidera un controllo più granulare sul learning rate e si sospetta che il modello possa beneficiare di periodi di apprendimento più lento.

- **NoSched**

Non applica alcuna modifica al learning rate durante l'addestramento, mantenendo il valore iniziale costante.

- **Loss function (`criterion`)**: è la funzione che misura quanto le predizioni del modello sono lontane dai target reali. Fornisce il segnale da minimizzare durante l'addestramento.

Nel problema di classificazione multiclasse spesso si usano:

- **CrossEntropy**

È la scelta standard per la **classificazione multiclasse**, che misura la differenza tra la distribuzione di probabilità predetta e la distribuzione reale delle etichette. Penalizza fortemente le predizioni molto sicure ma sbagliate, guidando il modello a migliorare le sue predizioni minimizzando questa differenza durante l'addestramento. È adatta quando il dataset è **abbastanza bilanciato** o quando non si vuole introdurre un trattamento diverso tra classi.

- **WeightedCrossEntropy**

Stessa idea della CrossEntropy standard, ma con `weight=class_weights`. Assegna pesi diversi a ciascuna classe in base alla loro frequenza nel training set. Le classi con meno campioni ricevono pesi più alti, aiutando il modello a prestare maggiore attenzione durante l'addestramento. È indicata per dataset con uno **sbilanciamento moderato**.

- **Focal**

La Focal Loss è progettata per affrontare dataset **fortemente sbilanciati**, riducendo il peso degli esempi facili e concentrandosi maggiormente su quelli difficili o mal classificati. È utile quando il modello tende a favorire troppo le classi più frequenti. I pesi vengono calcolati nello stesso modo della Weighted CrossEntropy.

È necessario definire due parametri: `alpha` serve a bilanciare le classi, mentre `gamma` è un parametro di focalizzazione regolabile che determina quanto velocemente gli esempi facili vengono ridotti di peso.

Setup degli iperparametri del modello

```
In [12]: importlib.reload(constants)
from constants import (
    EXEC_MODE_TRAIN,
    LEARNING_RATE, EPOCHS,
    WEIGHT_DECAY, EPSILON, MOMENTUM,
    MAX_LR, START_FACTOR, STEP_SIZE, STEPLR_GAMMA,
    ALPHA, FOCAL_LOSS_GAMMA
)

# #####

SELECTED_OPTIMIZER = "AdamW"
SELECTED_SCHEDULER = "OneCycleSched"
SELECTED_LOSS = "WeightedCrossEntropy"

if not EXEC_MODE_TRAIN:
    print(f"EXEC_MODE_TRAIN impostato su False.\n")
    print("La selezione della loss function e dell'optimizer è irrilevante "
          "in questa modalità.")
    pass

elif EXEC_MODE_TRAIN:

    # #####
    # Selezione dell'OPTIMIZER

    if SELECTED_OPTIMIZER == 'SGD':
        optimizer = torch.optim.SGD(
            model.parameters(),
            lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY, momentum=MOMENTUM
        )

    elif SELECTED_OPTIMIZER == 'Adam':
        optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

    elif SELECTED_OPTIMIZER == 'RMSprop':
```

```

optimizer = torch.optim.RMSprop(
    model.parameters(),
    lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY, momentum=MOMENTUM
)

elif SELECTED_OPTIMIZER == 'AdamW':
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY, eps=EPSILON
    )

else:
    raise ValueError(
        f"Tipo di optimizer non supportato: {SELECTED_OPTIMIZER}. "
        "Usare: 'Adam', 'AdamW', 'SGD' oppure 'RMSprop'."
    )

print(f"Optimizer selezionato: {SELECTED_OPTIMIZER}")

# #####
# Selezione dello SCHEDULER

if SELECTED_SCHEDULER == 'OneCycleSched':
    lr_sched = torch.optim.lr_scheduler.OneCycleLR(
        optimizer,
        max_lr=MAX_LR, steps_per_epoch=len(train_loader), epochs=EPOCHS
    )

elif SELECTED_SCHEDULER == 'LinearSched':
    lr_sched = torch.optim.lr_scheduler.LinearLR(
        optimizer,
        start_factor=START_FACTOR, total_iters=EPOCHS
    )

elif SELECTED_SCHEDULER == 'StepSched':
    lr_sched = torch.optim.lr_scheduler.StepLR(
        optimizer,
        step_size=STEP_SIZE, gamma=STEPLR_GAMMA
    )

elif SELECTED_SCHEDULER == 'NoSched':
    lr_sched = None

else:
    raise ValueError(
        f"Tipo di scheduler non supportato: {SELECTED_SCHEDULER}. "
        "Usare: 'OneCycleSched', 'LinearSched', 'StepSched' oppure 'NoSched'."
    )

print(f"Scheduler selezionato: {SELECTED_SCHEDULER}")

# #####
# Selezione della LOSS FUNCTION

loss_dtype = torch.float if DEVICE.type == 'mps' else torch.double

if SELECTED_LOSS == "CrossEntropy":
    criterion = nn.CrossEntropyLoss()

elif SELECTED_LOSS == "WeightedCrossEntropy":
    class_weights = compute_class_weights(
        y_train,
        NUM_CLASSES
    ).to(DEVICE).to(loss_dtype)

    criterion = nn.CrossEntropyLoss(weight=class_weights)

elif SELECTED_LOSS == "Focal":
    alpha = ALPHA

    if alpha == "class_weights":
        alpha = compute_class_weights(y_train, NUM_CLASSES)
    elif alpha == "uniform":
        alpha = torch.ones(NUM_CLASSES)
    elif alpha == "custom":
        alpha = torch.tensor(alpha)
    else:
        raise ValueError(
            f"Tipo di alpha non supportato: {alpha}. "
            "Usare: 'class_weights', 'uniform', oppure 'custom'."
        )

    alpha = alpha.to(DEVICE).to(loss_dtype)

    criterion = torch.hub.load(
        'adeelh/pytorch-multi-class-focal-loss',

```

```

        model='focal_loss',
        alpha=alpha,
        gamma=FOCAL_LOSS_GAMMA,
        reduction='mean',
        device=DEVICE,
        dtype=loss_dtype,
        force_reload=False
    )

else:
    raise ValueError(
        f"Tipo di loss non supportato: {SELECTED_LOSS}. "
        "Usare: 'CrossEntropy', 'WeightedCrossEntropy' oppure 'Focal'."
    )

print(f"Loss selezionata: {SELECTED_LOSS}")

```

Optimizer selezionato: AdamW
 Scheduler selezionato: OneCycleSched
 Loss selezionata: WeightedCrossEntropy

Training loop

Dopo aver configurato l'ottimizzatore e la loss function, il training procede per un numero definito di epoche. In ogni epoca, il modello viene addestrato sui batch del training set e valutato sul validation set.

Le principali metriche monitorate durante il training sono:

- **Loss:**

Misura l'errore del modello. L'obiettivo è minimizzare questa metrica.

- **Accuracy:**

Misura la percentuale di predizioni corrette. L'obiettivo è massimizzare questa metrica, ma può essere meno informativa in presenza di dataset sbilanciati.

- **Time:**

Misura il tempo impiegato per completare l'epoca, utile per stimare la durata complessiva del training e per ottimizzare la configurazione degli iperparametri.

```

In [ ]: importlib.reload(constants)
from constants import (
    EXEC_MODE_TRAIN,
    EPOCHS, PATIENCE, EARLY_STOPPING
)

# #####

if not EXEC_MODE_TRAIN:
    print(f"EXEC_MODE_TRAIN impostato su False.\n")
    print("Il modello non verrà addestrato.")
    pass

elif EXEC_MODE_TRAIN:
    # Questo dizionario terrà traccia delle metriche
    # di addestramento e validazione per ogni epoca.
    history = {
        'epoch': [],
        'time': [],
        'accuracy': [],
        'val_accuracy': [],
        'loss': [],
        'val_loss': [],
    }

    # Inizializza il timestamp per il salvataggio dei risultati e del modello.
    id = datetime.datetime.now().strftime("%Y-%m-%d_%H-%M")

    # Inizializza le variabili per il monitoraggio della miglior loss di validazione.
    best_val_loss = float('inf')
    best_model_wts = copy.deepcopy(model.state_dict())
    patience_counter = 0

    # #####

    # INIZIO TRAINING LOOP

    start_time = time.time()

    print(
        f"[INFO] Addestramento iniziato.\n" +
        f"[DATETIME] {id}\n" +

```

```

        f"[NAME] {model.get_model_name()}"
    )

    for epoch in range(EPOCHS):

        # Salva il timestamp di inizio epoca per calcolare la durata dell'epoca corrente.
        start_epoch_time = time.time()

        # Calcola la loss e l'accuratezza per il training set e il validation set.
        train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer)
        val_loss, val_acc = evaluate(model, val_loader, criterion)

        # Calcola la durata dell'epoca corrente.
        end_epoch_time = time.time()
        epoch_duration = end_epoch_time - start_epoch_time

        # Aggiorna lo storico delle metriche per l'epoca corrente.
        history['epoch'].append(epoch + 1)
        history['time'].append(epoch_duration)
        history['loss'].append(train_loss)
        history['accuracy'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_accuracy'].append(val_acc)

        # Stampa in console le metriche dell'epoca corrente.
        print(f"Epoch {epoch+1}/{EPOCHS} | "
              f"Loss: {train_loss:.4f} - Acc: {train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f} - Val Acc: {val_acc:.4f}"
              f" | Time: {epoch_duration:.2f}s")

        # CHECK : se la loss di validazione migliora,
        # salva il modello e resetta il contatore per la patience.
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_model_wts = copy.deepcopy(model.state_dict())
            patience_counter = 0
            print(f" -> Validation loss improved. Model saved.")
        else:
            patience_counter += 1
            print(f" -> No improvement. ", end="")
            print(f"Patience: {patience_counter} of {PATIENCE}" if True else "")

        # Se il contatore di patience raggiunge il limite
        # e l'early stopping è abilitato, si interrompe il training.
        if patience_counter >= PATIENCE and EARLY_STOPPING:
            print("Early stopping triggered.")
            break

    # end for epoch

    # Calcola e stampa il tempo totale di addestramento.
    total_time = time.time() - start_time
    print(f"\nTraining complete in {total_time/60:.2f} minutes.")

    # Salva i pesi del modello con la miglior loss di validazione.
    last_model = copy.deepcopy(model)
    model.load_state_dict(best_model_wts)

```

```
[INFO] Addestramento iniziato.
[DATETIME] 2026-08-24_18-05
[NAME] ResNet_C40
Epoch 1/30 | Loss: 3.4469 - Acc: 8.8102 | Val Loss: 2.9832 - Val Acc: 23.0659 | Time: 52.16s
-> Validation loss improved. Model saved.
Epoch 2/30 | Loss: 2.5441 - Acc: 31.7112 | Val Loss: 2.2486 - Val Acc: 41.3949 | Time: 47.58s
-> Validation loss improved. Model saved.
Epoch 3/30 | Loss: 1.9096 - Acc: 47.5433 | Val Loss: 1.8034 - Val Acc: 50.1309 | Time: 47.78s
-> Validation loss improved. Model saved.
Epoch 4/30 | Loss: 1.4736 - Acc: 57.7796 | Val Loss: 1.6191 - Val Acc: 56.0819 | Time: 47.81s
-> Validation loss improved. Model saved.
Epoch 5/30 | Loss: 1.1543 - Acc: 65.0932 | Val Loss: 1.4832 - Val Acc: 58.8431 | Time: 50.13s
-> Validation loss improved. Model saved.
Epoch 6/30 | Loss: 0.8785 - Acc: 71.5550 | Val Loss: 1.3285 - Val Acc: 64.1514 | Time: 54.27s
-> Validation loss improved. Model saved.
Epoch 7/30 | Loss: 0.6631 - Acc: 76.7413 | Val Loss: 1.3037 - Val Acc: 65.9843 | Time: 56.87s
-> Validation loss improved. Model saved.
Epoch 8/30 | Loss: 0.4755 - Acc: 82.2619 | Val Loss: 1.2520 - Val Acc: 68.7455 | Time: 57.49s
-> Validation loss improved. Model saved.
Epoch 9/30 | Loss: 0.3372 - Acc: 87.1564 | Val Loss: 1.2682 - Val Acc: 69.4597 | Time: 57.51s
-> No improvement. Patience: 1 / 7
Epoch 10/30 | Loss: 0.2554 - Acc: 90.2673 | Val Loss: 1.4009 - Val Acc: 68.7455 | Time: 55.98s
-> No improvement. Patience: 2 / 7
Epoch 11/30 | Loss: 0.1905 - Acc: 92.9829 | Val Loss: 1.3873 - Val Acc: 69.1264 | Time: 55.15s
-> No improvement. Patience: 3 / 7
Epoch 12/30 | Loss: 0.1280 - Acc: 95.2796 | Val Loss: 1.4175 - Val Acc: 71.2449 | Time: 53.82s
-> No improvement. Patience: 4 / 7
Epoch 13/30 | Loss: 0.0901 - Acc: 96.8091 | Val Loss: 1.4969 - Val Acc: 70.6498 | Time: 52.98s
-> No improvement. Patience: 5 / 7
Epoch 14/30 | Loss: 0.0678 - Acc: 97.6562 | Val Loss: 1.4947 - Val Acc: 71.6972 | Time: 52.11s
-> No improvement. Patience: 6 / 7
Epoch 15/30 | Loss: 0.0615 - Acc: 97.9480 | Val Loss: 1.4896 - Val Acc: 72.6018 | Time: 52.24s
-> No improvement. Patience: 7 / 7
Early stopping triggered.
```

Training complete in 13.23 minutes.

Salvataggio del modello

```
In [14]: importlib.reload(constants)
from constants import EXEC_MODE_TRAIN

# #####

if not EXEC_MODE_TRAIN:
    print(f"EXEC_MODE_TRAIN impostato su False.\n")
    print("Non è stato eseguito alcun addestramento. Il modello non verrà salvato.")
    pass

elif EXEC_MODE_TRAIN:

    model_name = \
        f"{len(history['loss'])}E_" \
        f"{SELECTED_OPTIMIZER}_" \
        f"{SELECTED_SCHEDULER}_" \
        f"{SELECTED_LOSS}_" \
        f"{SELECTED_MODEL}"

    output_dir = f"../results/{id}/"

    os.makedirs(f"{output_dir}/{model_name}", exist_ok=True)

    # with open(f"{output_dir}/{model_name}/model_summary.txt", "w") as f:
    #     model.summary(print_fn=lambda x: f.write(x + "\n"))

    # Salva lo storico delle metriche di addestramento e validazione in un file CSV.
    df_history = pd.DataFrame(history)
    df_history.to_csv(f"{output_dir}/{model_name}/training_history.csv", index=False)

    # Salva i pesi del modello addestrato in un file .PTH
    torch.save(model.state_dict(), f"{output_dir}/{model_name}/model.pth")
```

Fase di valutazione

Dopo la fase di training, il modello viene valutato sul test set per misurare le sue prestazioni su dati mai visti prima.

Le principali metriche calcolate includono:

- **Loss:**

Misura l'errore del modello sul test set.

- **Accuracy:**

Misura la percentuale di predizioni corrette sul test set.

Si completa il notebook con la visualizzazione dei risultati finali e la generazione di report dettagliati. Questi includono grafici della funzione di loss e di accuracy durante le fasi di training e validazione, la matrice di confusione normalizzata sul test set, e un report di classificazione con precision, recall, F1-score e support per ciascuna classe.

Valutazione dell'accuracy sul test set

```
In [15]: importlib.reload(constants)
from constants import EXEC_MODE_TRAIN

# #####

if not EXEC_MODE_TRAIN:
    print(f"EXEC_MODE_TRAIN impostato su False.\n")
    print("Non è stato eseguito alcun addestramento. Impossibile valutare il modello.")
    pass

elif EXEC_MODE_TRAIN:
    # Calcola la loss e l'accuratezza per il test set,
    # per valutare le prestazioni del modello su dati mai visti prima.
    test_loss, test_acc = evaluate(model, test_loader, criterion)
    print(f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.2f}%")
```

Test Loss: 1.2520 | Test Acc: 68.80%

Grafici di loss e accuracy su training / validation set

```
In [56]: importlib.reload(constants)
from constants import SAVE_OUTPUT

# #####

# Recupero dello storico delle metriche di addestramento e validazione.
loss = history['loss']
val_loss = history['val_loss']
acc = history['accuracy']
val_acc = history['val_accuracy']

# Calcolo dell'epoca con validation loss minima per evidenziarla nel grafico.
best_epoch = int(np.argmin(val_loss)) + 1

# #####

plots = plt.figure(figsize=(12, 5))

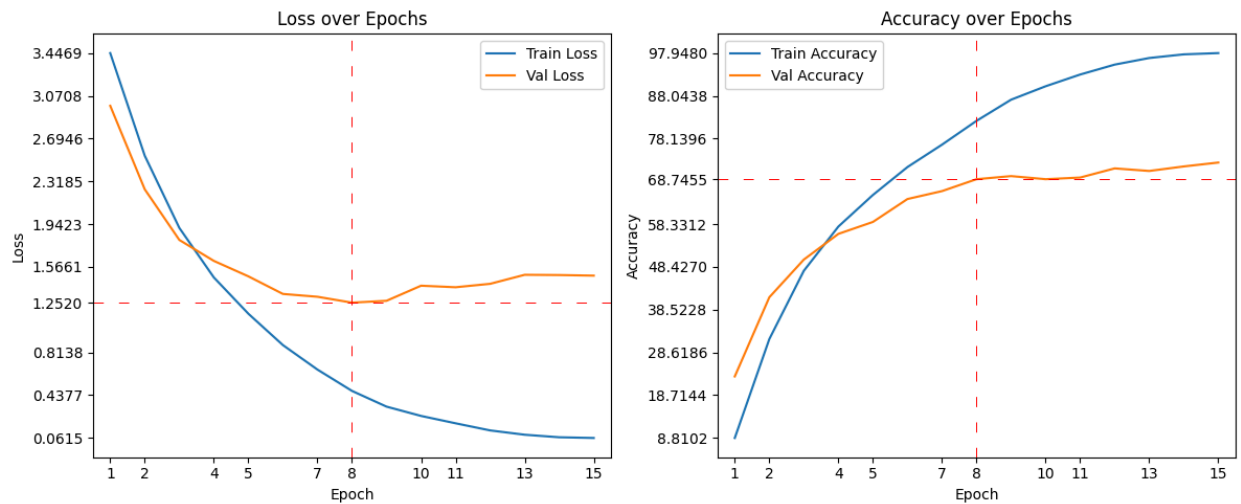
loss_plot = plt.subplot(1, 2, 1)
draw_train_val_plot(loss_plot, 'Loss', loss, val_loss, best_epoch)

acc_plot = plt.subplot(1, 2, 2)
draw_train_val_plot(acc_plot, 'Accuracy', acc, val_acc, best_epoch)

plt.tight_layout()
plt.show()

# #####

if SAVE_OUTPUT:
    plots.savefig(f"{output_dir}/{model_name}/train_val_plots.png")
```



Matrice di confusione

Una matrice di confusione è una tabella che riassume le prestazioni di un modello di classificazione, mettendo a confronto le classi reali (righe) con le classi predette dal modello (colonne). Ogni cella (i, j) indica quante osservazioni della classe reale i sono state classificate come classe predetta j , permettendo di vedere non solo quante previsioni sono corrette, ma anche quali errori specifici il modello commette.

```
In [35]: importlib.reload(constants)
from constants import SAVE_OUTPUT

# ##### #

model.eval()
all_preds = []
all_labels = []

# Si utilizza "torch.no_grad()" per disabilitare
# il calcolo del gradiente durante la fase di valutazione,
# migliorando le prestazioni e riducendo l'uso della memoria.
with torch.no_grad():
    for inputs, labels in test_loader:

        # Sposta i dati sul DEVICE selezionato prima della predizione.
        inputs = inputs.to(DEVICE)

        # Calcola le predizioni del modello per il batch corrente.
        outputs = model(inputs)
        _, predicted = outputs.max(1)

        # Infine, si utilizza "extend" per aggiungere le predizioni e le etichette
        # del batch corrente alle liste globali "all_preds" e "all_labels".
        # A differenza di "append", che aggiunge un singolo elemento,
        # "extend" aggiunge tutti gli elementi di un Iterable alla lista esistente.
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

        # ATTENZIONE!
        # Sostituire con "all_labels.extend(labels.numpy())" se si utilizza la CPU.

    # end for inputs, labels

cm = confusion_matrix(all_labels, all_preds, normalize='true')

confusion_matrix_plot = plt.figure(figsize=(10, 8))

ax = sns.heatmap(
    cm,
    annot=False,
    fmt='',
    cmap='plasma_r',  # Mappa colori plasma invertita
    linewidths=0.5,   # Spessore linee della griglia
    mask= cm == 0,    # Applica la maschera
    linecolor='black', # Colore linee della griglia
    square=True,      # Celle quadrate
    cbar_kws={ "ticks": [0.1, 1, 10, 100] },
    xticklabels=le.classes_,
    yticklabels=le.classes_
)

ax.set_facecolor('white')
```



```

empty_cols = np.where(cm.sum(axis=0) == 0)[0]

# Se ci sono colonne vuote, aggiunge un simbolo '.' al centro di ciascuna cella vuota.
for col in empty_cols:
    for row in range(cm.shape[0]):

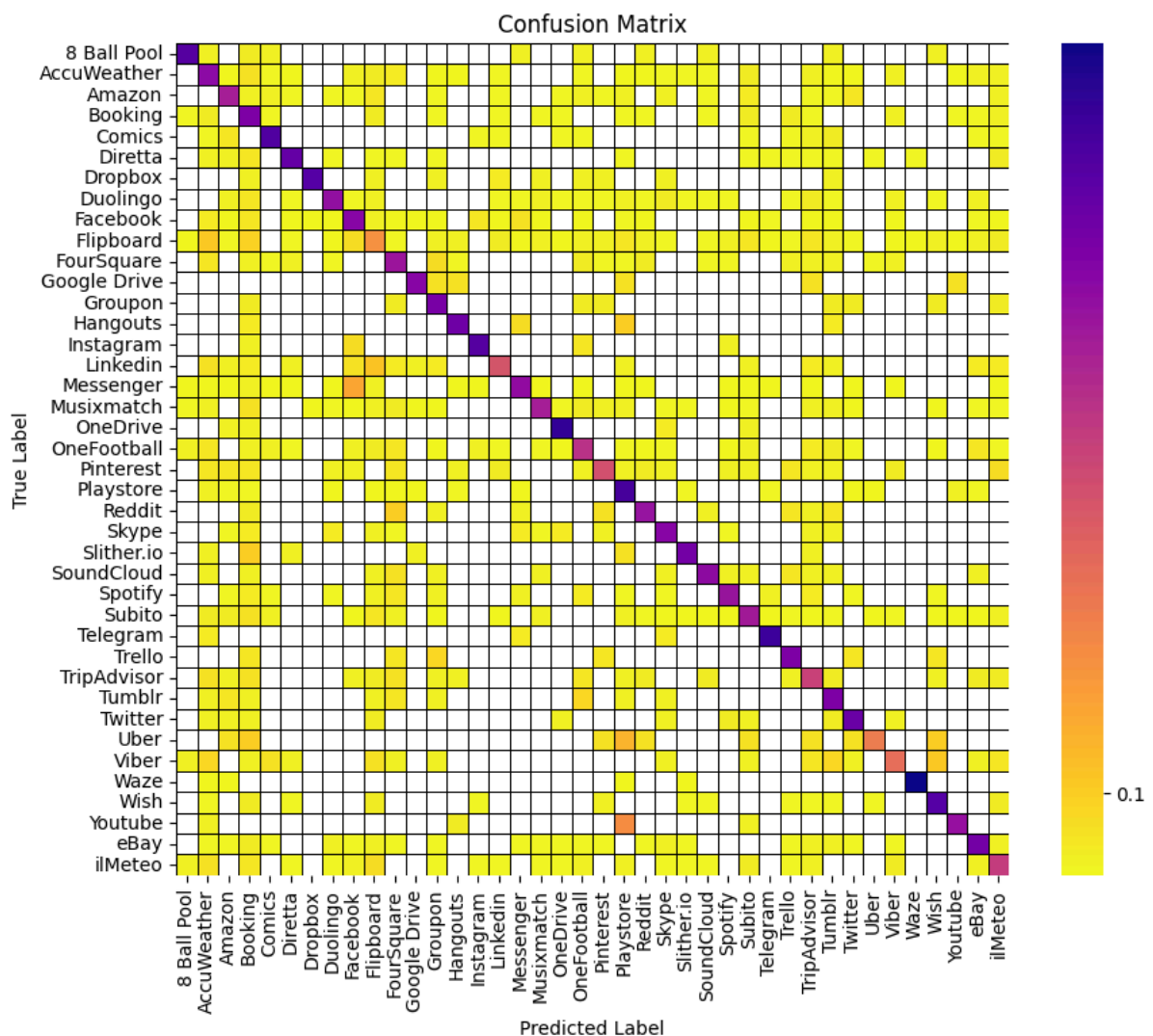
        # Per centrare il testo nella cella, si aggiunge 0.5 a "row" e "col".
        ax.text(
            col + 0.5, row + 0.5, '.',
            ha='center', va='center',
            color='black', fontsize=15
        )

        # end for row
    # end for col

plt.title("Confusion Matrix")
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

if SAVE_OUTPUT:
    confusion_matrix_plot.savefig(f"{output_dir}/{model_name}/confusion_matrix.png")

```



Report di classificazione completo

```

In [36]: importlib.reload(constants)
from constants import SAVE_OUTPUT

# #####

report_dict = classification_report(
    all_labels, all_preds,
    target_names=le.classes_, labels=np.arange(NUM_CLASSES),
    digits=4, zero_division=0
)

if SAVE_OUTPUT:

```

```
with open(f"{output_dir}/{model_name}/classification_report.txt", "w") as f:  
    f.write(report_dict)
```
